SE3308 Algorithm Design I — Assignment 2

Exercises 2.14, 2.19, 2.22, 3.7, 3.11, 3.28

Xilang Chen 524031910668

Throughout, log denotes $\log_2$. Graphs are given by adjacency lists, with $n = |V|$ and $m = |E|$.

Exercise 2.14

We assume the elements can be compared (any total order works, e.g. the order of their binary encodings).

Algorithm. Sort the array with mergesort, then scan it once and keep an element only if it differs from its predecessor.

Algorithm 1 Removing duplicates

```
1: function RemoveDuplicates(A[1..n])
2:     A ← MergeSort(A)
3:     B ← empty list
4:     for i ← 1 to n do
5:         if i = 1 or A[i] ≠ A[i − 1] then
6:             append A[i] to B
7:         end if
8:     end for
9:     return B
10: end function
```

Correctness. After sorting, equal elements are contiguous: if $A[i] = A[j]$ with $i < j$, then for every $i \le l \le j$ we have $A[i] \le A[l] \le A[j] = A[i]$, so $A[l] = A[i]$. Hence the sorted array consists of maximal runs, each run containing all copies of one distinct value. The scan appends exactly the first element of each run (the first element of a run either is $A[1]$ or differs from its predecessor, which belongs to the previous run; every other element of a run equals its predecessor). Therefore B contains every distinct value of the input exactly once.

Running time. Mergesort takes $O(n \log n)$ and the scan takes $O(n)$, so the total is $\boxed{O(n \log n)}$.

Remark. If the original order of the surviving elements must be kept, sort the pairs $(A[i], i)$ lexicographically instead, keep the first pair of each run (it carries the smallest index of that value), and finally sort the survivors by index. This is still $O(n \log n)$.

Exercise 2.19

Recall that MERGE combines two sorted arrays of total length L in $O(L)$ time.

(a) After merging the first i arrays we hold a sorted array of in elements; merging in the $(i+1)$-st array costs $\Theta(in+n) = \Theta((i+1)n)$. The total time is

$$\sum_{i=1}^{k-1} \Theta\big((i+1)n\big) = \Theta\Big(n \sum_{j=2}^{k} j\Big) = \Theta\Big(n\Big(\tfrac{k(k+1)}{2} - 1\Big)\Big) = \boxed{\Theta(k^2 n)}.$$

(b) Divide and conquer: split the k arrays into two groups of $\lceil k/2 \rceil$ and $\lfloor k/2 \rfloor$ arrays, merge each group recursively, and merge the two results.

Algorithm 2 k-way merge

```
1: function MultiMerge(A_1, ..., A_k)
2:     if k = 1 then
3:         return A_1
4:     end if
5:     h ← ⌈k/2⌉
6:     L ← MultiMerge(A_1, ..., A_h)
7:     R ← MultiMerge(A_{h+1}, ..., A_k)
8:     return Merge(L, R)
9: end function
```

Correctness follows by induction on k: for $k = 1$ the single array is already sorted; for $k \geq 2$, by induction L and R are sorted and together contain all kn elements, and MERGE of two sorted arrays is sorted.

Running time. Let $T(k)$ be the time for k arrays of n elements each. The top-level merge handles kn elements, so

$$T(k) = T(\lceil k/2 \rceil) + T(\lfloor k/2 \rfloor) + O(kn), \qquad T(1) = O(1).$$

In the recursion tree, the subproblems on any one level involve disjoint groups of the original arrays, so the merges on that level handle at most kn elements in total and cost $O(kn)$. Group sizes halve (rounding up) at each level, so there are $\lceil \log k \rceil$ levels with merges. Hence

$$T(k) = \boxed{O(kn \log k)},$$

an improvement by a factor of $k/\log k$ over part (a).

Exercise 2.22

Let the sorted lists be $A[1..m]$ and $B[1..n]$, stored as arrays (constant-time access by index), and let $1 \leq k \leq m+n$. For convenience set $A[0] = B[0] = -\infty$ and $A[m+1] = B[n+1] = +\infty$.

Idea. The k smallest elements of the union consist of a prefix $A[1..i]$ of A and a prefix $B[1..j]$ of B with $i + j = k$. We binary-search for the right i. The feasible range is

$$\ell = \max(0,\, k-n) \;\leq\; i \;\leq\; h = \min(k,\, m),$$

because $0 \le i \le m$ and $0 \le j = k - i \le n$. For i in this range define the predicate

$$P(i): \quad i = \ell \ \text{ or } \ A[i] \le B[k-i+1].$$

Algorithm 3 k-th smallest element of $A \cup B$

```
1: function SELECT2(A[1..m], B[1..n], k)
2:     ℓ ← max(0, k − n);   h ← min(k, m)
3:     while ℓ < h do                ▷ invariant: P(ℓ) holds and the largest i with P(i) lies in [ℓ, h]
4:         i ← ⌈(ℓ + h)/2⌉                                                          ▷ ℓ < i ≤ h
5:         if A[i] ≤ B[k − i + 1] then
6:             ℓ ← i
7:         else
8:             h ← i − 1
9:         end if
10:    end while
11:    i ← ℓ;   j ← k − i
12:    return max(A[i], B[j])
13: end function
```

Inside the loop $\ell < i \le h$ implies $1 \le i \le m$ and $1 \le k - i + 1 \le n$ (since $i \ge \ell + 1 \ge k - n + 1$ and $i \le k$), so both indices refer to real elements.

Correctness. *Monotonicity.* As i grows, $A[i]$ does not decrease and $B[k-i+1]$ does not increase. So if $A[i] \le B[k-i+1]$ fails for some i, it fails for all larger i. Hence P is true on an initial segment $[\ell, i^*]$ of the range and false afterwards, and the binary search returns $i = i^*$, the largest i with $P(i)$ (each step keeps $P(\ell)$ true and discards only indices where P is false).

The split is correct. Let $i = i^*$ and $j = k - i$. We claim

$$(1)\ A[i] \le B[j+1] \qquad \text{and} \qquad (2)\ B[j] \le A[i+1].$$

For (1): if $i > \ell$ this is $P(i)$. If $i = \ell = 0$ then $A[0] = -\infty$; if $i = \ell = k - n$ then $j = n$ and $B[n+1] = +\infty$. For (2): if $i < h$, then $P(i+1)$ is false, i.e. $A[i+1] > B[k-i] = B[j]$. If $i = h = m$ then $A[m+1] = +\infty$; if $i = h = k$ then $j = 0$ and $B[0] = -\infty$.

Now let $X = A[1..i] \cup B[1..j]$ (k elements) and $Y = A[i+1..m] \cup B[j+1..n]$. Every element of X is at most every element of Y: within one array this is sortedness, and across arrays $A[1..i] \le A[i] \le B[j+1] \le B[j+1..n]$ by (1) and $B[1..j] \le B[j] \le A[i+1] \le A[i+1..m]$ by (2). Therefore X is a set of k smallest elements of the union, and the k-th smallest element is the maximum of X, which is $\max(A[i], B[j])$ (at least one of i, j is positive because $k \ge 1$, so the maximum is finite).

Running time. Each iteration does $O(1)$ work and roughly halves $h - \ell$, which is initially at most $\min(m, n, k)$. Hence the time is

$$O\big(1 + \log \min(m, n)\big) \subseteq \boxed{O(\log m + \log n)}.$$

Exercise 3.7

(a) Run DFS and give each newly discovered vertex the color opposite to its parent's. The graph is bipartite iff no edge joins two vertices of the same color.

Algorithm 4 Testing bipartiteness

```
1: function BIPARTITE(G = (V, E))
2:     for each v ∈ V do
3:         color(v) ← NIL
4:     end for
5:     for each s ∈ V do
6:         if color(s) = NIL then
7:             color(s) ← 0;  EXPLORE(s)
8:         end if
9:     end for
10:    return YES                          ▷ V_1 = {v : color(v) = 0}, V_2 = {v : color(v) = 1}
11: end function

12: procedure EXPLORE(u)
13:    for each edge {u, v} ∈ E do
14:        if color(v) = NIL then
15:            color(v) ← 1 − color(u);  EXPLORE(v)
16:        else if color(v) = color(u) then
17:            halt and return NO
18:        end if
19:    end for
20: end procedure
```

Running time. This is DFS with $O(1)$ extra work per vertex and per adjacency-list entry: $O(n+m)$.

Correctness. If the algorithm answers YES, every vertex is colored, and every edge $\{u, v\}$ was examined from u when EXPLORE(u) ran, at which point v was either colored now with the opposite color or already colored and checked to differ. So every edge joins V_1 and V_2: the graph is bipartite.

If the algorithm answers NO, it found an edge $\{u, v\}$ with $color(u) = color(v)$, and u, v lie in the same DFS tree T (rooted at some s). Suppose for contradiction that G had a bipartition, i.e. a function $c : V \to \{0, 1\}$ with $c(x) \neq c(y)$ for every edge $\{x, y\}$. Both c and $color$ change value across every tree edge of T, so by induction along tree paths from s, $c(x) = c(s) \oplus color(x)$ for all $x \in T$. Then $color(u) = color(v)$ gives $c(u) = c(v)$, contradicting that $\{u, v\}$ is an edge. So NO is correct.

(b) *(⇒)* Let $V = V_1 \cup V_2$ be a bipartition and let $v_0, v_1, \ldots, v_{L-1}, v_0$ be a cycle. Consecutive vertices are on opposite sides, so by induction v_i is on the side of v_0 iff i is even. The edge $\{v_{L-1}, v_0\}$ forces v_{L-1} to the other side, so $L - 1$ is odd and L is even.

(⇐) Suppose G has no odd cycle. Treat each connected component separately. Pick a root r and a spanning tree T of the component (e.g. the DFS tree), and color each vertex x by the parity of its depth $d(x)$ in T. Tree edges join depths differing by 1, so they are properly colored. Suppose some non-tree edge $\{u, v\}$ had $d(u) \equiv d(v) \pmod 2$. Let w be the lowest common ancestor of u and v in T. The tree paths $u \rightsquigarrow w$ and $w \rightsquigarrow v$ share only w, so together they form a simple path from u to v of length

$$(d(u) - d(w)) + (d(v) - d(w)) = d(u) + d(v) - 2d(w),$$

which is even, and positive since $u \neq v$. This path does not use the edge $\{u, v\}$ (that edge is not in

T), so adding $\{u, v\}$ closes a simple cycle of odd length $d(u)+d(v)-2d(w)+1 \geq 3$, a contradiction. Hence the parity coloring is a proper 2-coloring, i.e. G is bipartite.

(c) Three colors.

Three suffice. Let C be the unique odd cycle and $e = \{a, b\}$ an edge of C. Every cycle of $G - e$ is a cycle of G that avoids e, hence is not C, hence has even length. By (b), $G - e$ has a proper coloring with colors $1, 2$. Recolor a with a new color 3. Edges not incident to a are unaffected; edges incident to a, including e, now join color 3 to color 1 or 2. This is a proper 3-coloring of G.

Three are necessary. G contains an odd cycle, so by (b) it is not bipartite, i.e. not 2-colorable.

Exercise 3.11

Let $e = \{u, v\}$ (in a simple graph).

Claim. *G has a cycle containing e if and only if v is reachable from u in $G - e$.*

Proof. ($\Rightarrow$) A (simple) cycle through e uses e exactly once; deleting e from it leaves a path from u to v that avoids e. ($\Leftarrow$) Take a simple path P from u to v in $G - e$. Since e is the only edge between u and v and it was removed, P has length at least 2. Then P followed by e is a simple cycle (of length at least 3) containing e. □

Algorithm 5 Is there a cycle through $e = \{u, v\}$?

```
1: function CycleThrough(G, {u, v})
2:     mark all vertices unvisited
3:     Explore(u) in G, skipping the edge {u, v}      ▷ ignore v in u's list and u in v's list
4:     return YES if v is visited, else NO
5: end function
```

By the correctness of EXPLORE (it visits exactly the vertices reachable from u), and the claim, the answer is correct. One EXPLORE takes $O(n+m)$ time, and skipping e costs $O(1)$ per adjacency-list entry, so the algorithm is linear.

Exercise 3.28

Write $I_0 = (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_3) \wedge (x_1 \vee x_2) \wedge (\overline{x}_3 \vee x_4) \wedge (\overline{x}_1 \vee x_4)$ for the given instance.

(a) Yes, there is exactly one other. The clauses $(x_1 \vee \overline{x}_2)$ and $(x_1 \vee x_2)$ together force $x_1 = \text{true}$ (if x_1 were false they would require both $\overline{x}_2$ and x_2). Then $(\overline{x}_1 \vee \overline{x}_3)$ forces $x_3 = \text{false}$ and $(\overline{x}_1 \vee x_4)$ forces $x_4 = \text{true}$. With these values all five clauses hold regardless of x_2. Hence the satisfying assignments (x_1, x_2, x_3, x_4) are exactly

$$(\mathrm{T}, \mathrm{F}, \mathrm{F}, \mathrm{T}) \quad \text{and} \quad (\mathrm{T}, \mathrm{T}, \mathrm{F}, \mathrm{T}),$$

so the only other one is $x_1 = x_2 = x_4 = \text{true}$, $x_3 = \text{false}$.

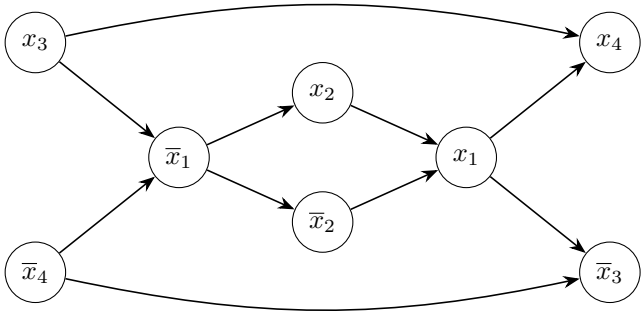
(b)

$$I_1 = (x_1 \vee x_2) \wedge (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee x_3) \wedge (\overline{x}_3 \vee x_4) \wedge (\overline{x}_4 \vee \overline{x}_1).$$

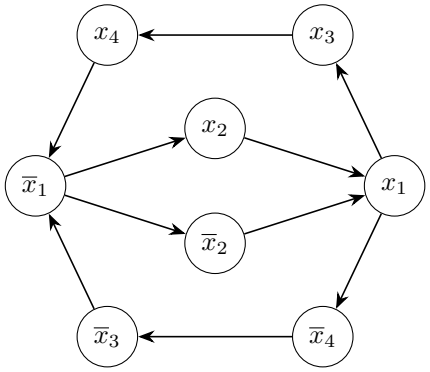
The first two clauses force $x_1 =$ true. Then the last three force $x_3 =$ true, then $x_4 =$ true, then $x_1 =$ false, a contradiction. So I_1 is unsatisfiable.

(c) Each clause $(\alpha \vee \beta)$ contributes the edges $\overline{\alpha} \to \beta$ and $\overline{\beta} \to \alpha$.

G_{I_0}			G_{I_1}		
$(x_1 \vee \overline{x}_2)$	$\overline{x}_1 \to \overline{x}_2$	$x_2 \to x_1$	$(x_1 \vee x_2)$	$\overline{x}_1 \to x_2$	$\overline{x}_2 \to x_1$
$(\overline{x}_1 \vee \overline{x}_3)$	$x_1 \to \overline{x}_3$	$x_3 \to \overline{x}_1$	$(x_1 \vee \overline{x}_2)$	$\overline{x}_1 \to \overline{x}_2$	$x_2 \to x_1$
$(x_1 \vee x_2)$	$\overline{x}_1 \to x_2$	$\overline{x}_2 \to x_1$	$(\overline{x}_1 \vee x_3)$	$x_1 \to x_3$	$\overline{x}_3 \to \overline{x}_1$
$(\overline{x}_3 \vee x_4)$	$x_3 \to x_4$	$\overline{x}_4 \to \overline{x}_3$	$(\overline{x}_3 \vee x_4)$	$x_3 \to x_4$	$\overline{x}_4 \to \overline{x}_3$
$(\overline{x}_1 \vee x_4)$	$x_1 \to x_4$	$\overline{x}_4 \to \overline{x}_1$	$(\overline{x}_4 \vee \overline{x}_1)$	$x_4 \to \overline{x}_1$	$x_1 \to \overline{x}_4$



G_{I_0}: acyclic, every SCC is a single literal



G_{I_1}: one SCC containing all 8 literals

In G_{I_0} no literal can reach itself (the left-to-right drawing is a topological order), so all eight SCCs are singletons and no SCC contains both x and $\overline{x}$. In G_{I_1}, $x_1 \to x_3 \to x_4 \to \overline{x}_1 \to x_2 \to x_1$, $\overline{x}_1 \to \overline{x}_2 \to x_1$ and $x_1 \to \overline{x}_4 \to \overline{x}_3 \to \overline{x}_1$ are cycles through x_1 covering all eight literals, so they form a single SCC, which contains x_1 and $\overline{x}_1$.

(d)

Lemma (Edges are implications)**.** *Under any satisfying assignment, if $\lambda \rightsquigarrow \mu$ is a path in G_I and λ is true, then μ is true.*

Proof. For a single edge $\overline{\alpha} \to \beta$ coming from clause $(\alpha \vee \beta)$: if $\overline{\alpha}$ is true then α is false, so β must be true for the clause to hold. The case of a path follows by induction on its length. □

Suppose x and $\overline{x}$ lie in the same SCC, so there are paths $x \rightsquigarrow \overline{x}$ and $\overline{x} \rightsquigarrow x$. In a satisfying assignment, if x is true the first path makes $\overline{x}$ true; if x is false then $\overline{x}$ is true and the second path makes x true. Both are contradictions, so I is unsatisfiable.

(e) Write $\overline{S} = \{\overline{\lambda} : \lambda \in S\}$ for a set of literals S.

Skew symmetry. The edges $\overline{\alpha} \to \beta$ and $\overline{\beta} \to \alpha$ always come in pairs, so $\lambda \to \mu$ is an edge iff $\overline{\mu} \to \overline{\lambda}$ is an edge; hence $\lambda \rightsquigarrow \mu$ iff $\overline{\mu} \rightsquigarrow \overline{\lambda}$. Consequently, if S is an SCC then so is $\overline{S}$.

The procedure. Let R be the set of literals not yet deleted, initially all $2n$ of them. While $R \neq \emptyset$: pick a sink SCC S of the induced subgraph $G_I[R]$, make every literal of S true and every literal of $\overline{S}$ false, and delete $S \cup \overline{S}$ from R.

Invariant: R is closed under negation and is a union of SCCs of G_I. This holds initially. If it holds, then the SCCs of $G_I[R]$ are exactly the SCCs of G_I inside R (a path between two vertices of one SCC C of G_I stays inside $C \subseteq R$, and conversely mutual reachability in $G_I[R]$ implies it in G_I). So the chosen S is an SCC of G_I, $\overline{S}$ is an SCC of G_I by skew symmetry, and $\overline{S} \subseteq R$ by closure. Removing $S \cup \overline{S}$ preserves both properties.

The assignment is well defined. By hypothesis S does not contain both λ and $\overline{\lambda}$, so $S \cap \overline{S} = \emptyset$ and no literal gets both values. Every step removes a variable together with its negation, and assigns all of them, so at the end every variable has exactly one value.

No edge goes from a true literal to a false literal. Suppose $\lambda \to \mu$ is an edge with λ true and μ false. Say λ was assigned at step t (so $\lambda \in S_t$) and μ at step t' (so $\overline{\mu} \in S_{t'}$).

- If $t \leq t'$: at the start of step t, μ is still in R. Since S_t is a sink of $G_I[R]$ and $\lambda \in S_t$, the edge $\lambda \to \mu$ forces $\mu \in S_t$, so μ is true, a contradiction.
- If $t' < t$: at the start of step t', λ and hence $\overline{\lambda}$ are in R. The edge $\overline{\mu} \to \overline{\lambda}$ exists by skew symmetry and $\overline{\mu} \in S_{t'}$, a sink, so $\overline{\lambda} \in S_{t'}$ and λ is false, a contradiction.

Every clause is satisfied. Take a clause $(\alpha \vee \beta)$. If α were false, then $\overline{\alpha}$ is true, and the edge $\overline{\alpha} \to \beta$ forces β to be true. So I is satisfiable.

(f) By (d) and (e), I is satisfiable iff no SCC of G_I contains a variable together with its negation. The algorithm:

1. Build G_I: $2n$ vertices and $2m$ edges, in $O(n+m)$ time.
2. Compute its SCCs with the linear-time two-pass DFS algorithm, in $O(n+m)$ time, labelling each literal with its component.
3. For each variable x, compare the labels of x and $\overline{x}$ ($O(n)$ in total). If some pair agrees, output "unsatisfiable".
4. Otherwise output "satisfiable". A satisfying assignment can also be produced: that algorithm outputs the SCCs sink-first, i.e. each component it outputs is a sink of the graph left after removing the earlier ones. Process the components in that order and make the literals of a component true whenever they are still unassigned (and their negations false). Each component handled this way is a sink of a graph containing the current $G_I[R]$, hence a sink of $G_I[R]$, so this is an instance of the procedure in (e). It costs $O(n)$.

The total running time is $O(n+m)$, linear in the size of the instance.